C PROGRAMMING

Lesson 4 — Basic Arithmetic

Based on: Bro Code C Tutorial Series

1. Basic Arithmetic Operators

C supports five basic arithmetic operators. Results are stored in a third variable.

```
int x = 10;
int y = 3;
int z;

z = x + y;   // Addition       --> z = 13
z = x - y;   // Subtraction    --> z = 7
z = x * y;   // Multiplication --> z = 30
z = x / y;   // Division       --> z = 3  (not 3.33!)
z = x % y;   // Modulus        --> z = 1  (remainder of 10 / 3)
```

2. Integer Division — The Big Gotcha

When you divide two integers in C, the decimal part is thrown away (truncated). This catches a lot of beginners off guard.

```
int x = 2;
int y = 3;
int z = x / y;
printf("%d\n", z);   // Output: 0  (not 0.666...!)

// Fix: make at least one operand a float or double
float a = 2;
float b = 3;
float c = a / b;
printf("%f\n", c);   // Output: 0.666667
```

> **Key Rule:** If both operands are int, the result is int — decimals are cut off, not rounded. To get a decimal result, use float or double.

3. Modulus Operator %

The modulus operator gives you the remainder after division. It only works with integers.

```
printf("%d\n", 10 % 3);   // Output: 1  (10 = 3*3 + 1)
printf("%d\n", 10 % 2);   // Output: 0  (10 = 2*5 + 0)
printf("%d\n", 7  % 4);   // Output: 3  (7  = 4*1 + 3)
```

Common use case — even or odd check

```
int number = 7;

// If remainder when divided by 2 is 0 --> even
// If remainder when divided by 2 is 1 --> odd
printf("%d\n", number % 2);   // Output: 1 --> odd
```

> **Tip:** Modulus is one of the most useful operators in C. You will use it constantly in loops and conditions.

4. Increment & Decrement Operators

These are shortcuts to increase or decrease a variable by exactly 1. You will see these everywhere — especially inside loops.

```
int x = 10;

x++;   // Same as x = x + 1  --> x is now 11
x--;   // Same as x = x - 1  --> x is now 10 again

printf("%d\n", x);   // Output: 10
```

- **x++** increments after the current statement uses x (post-increment)
- **x--** decrements after the current statement uses x (post-decrement)
- **Most common use:** counting iterations in loops

5. Augmented Assignment Operators

These are shorthand operators that update a variable using its current value. They make your code shorter and easier to read.

```
int x = 10;

x += 2;   // Same as x = x + 2  --> x = 12
x -= 3;   // Same as x = x - 3  --> x = 9
x *= 2;   // Same as x = x * 2  --> x = 18
x /= 3;   // Same as x = x / 3  --> x = 6
x %= 4;   // Same as x = x % 4  --> x = 2
```

> **Good Practice:** Always use augmented assignment operators instead of writing x = x + 2. It is cleaner and the standard way to write C code.

6. Practice Problem

Practice: Calculator Program

Write a C program that acts as a simple calculator for two numbers. Use x = 15 and y = 4.

Requirements:

- Declare x and y as integers
- Print the result of all 5 basic operations: +, -, *, /, %
- For division, also print the float result (declare separate float variables)
- Then increment x by 1 and print the new value
- Finally, use x += 10 and print the result

Expected Output

```
15 + 4  = 19
15 - 4  = 11
15 * 4  = 60
15 / 4  = 3        (int division)
15 / 4  = 3.750000 (float division)
15 % 4  = 3
x after x++ = 16
x after x += 10 = 26
```

Hint: For float division, either declare x and y as float, or cast one of them: (float)x / y